

20MCA281 - Internet of Things

Module 3

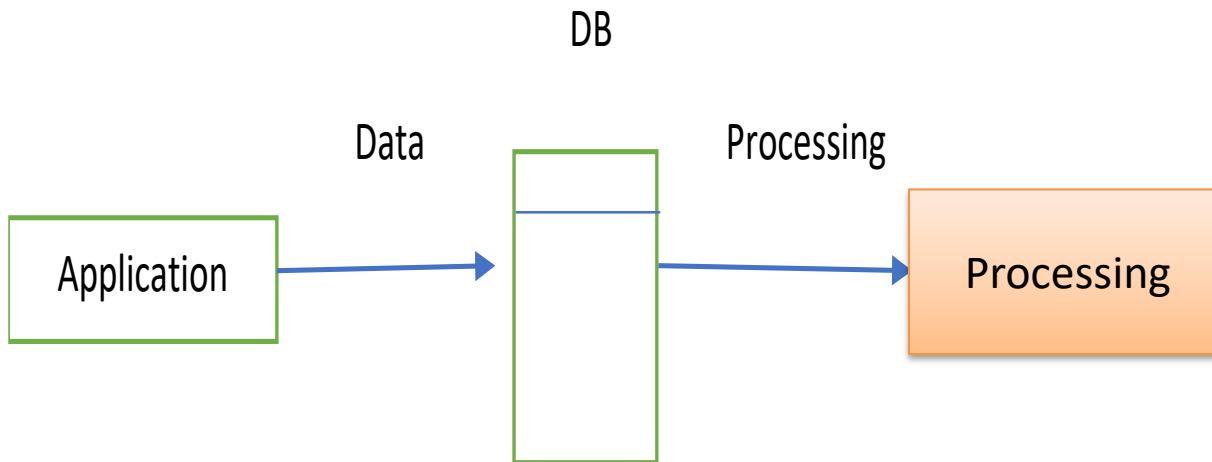
Stream Processing in IoT

Introduction

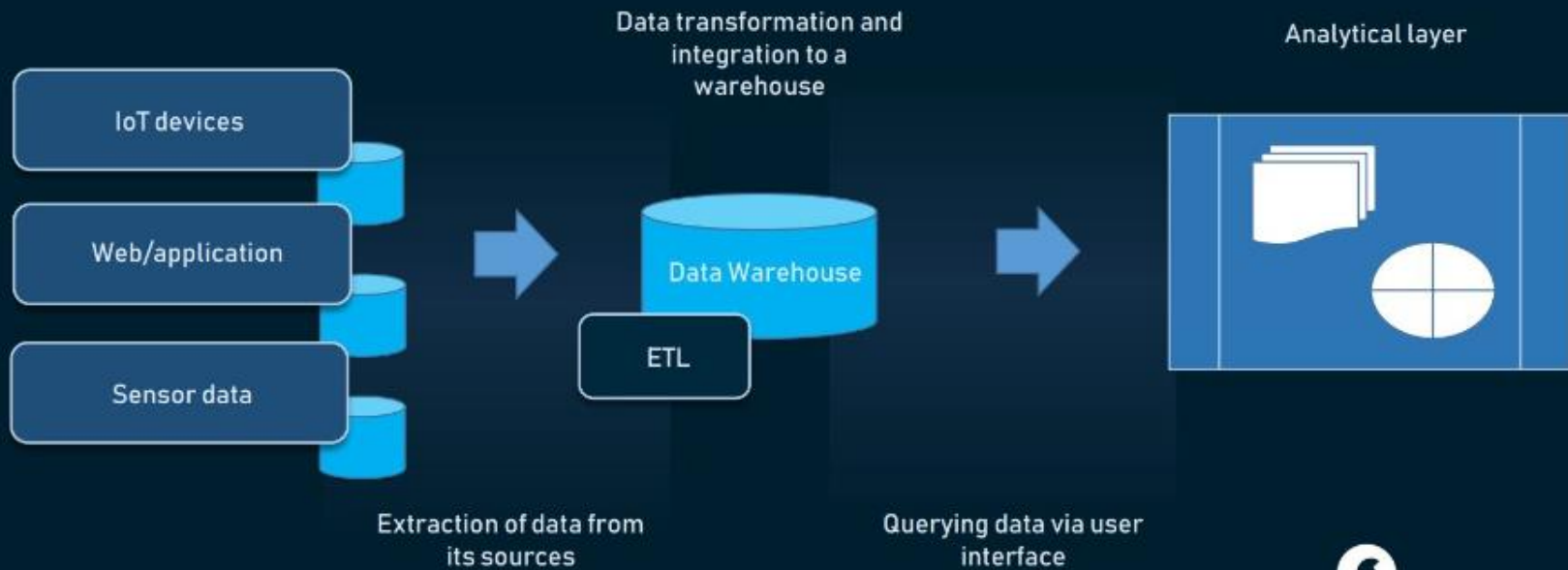
- Driven by the incompetence of the traditional batch-processing paradigm
- Building a modern information-technology system demands the ability of:
 1. Processing an unprecedented volume of data using distributed resources, and
 2. Exploring the concealed value of data within a tight time-constraint

- MapReduce is a highly scalable and widely adopted programming model which moves the computing power to the vicinity of data so that the enormous processing target can be divided and conquered
- Batch processing systems are struggling to meet the second goal (storing the data somewhere before executing relevant operations against it) to meet strict time-constraints

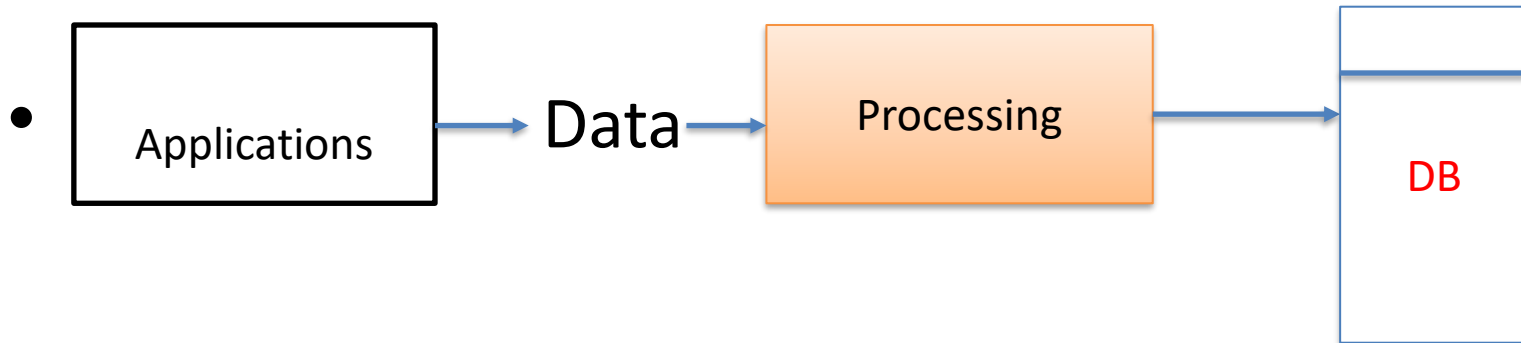
Batch Processing



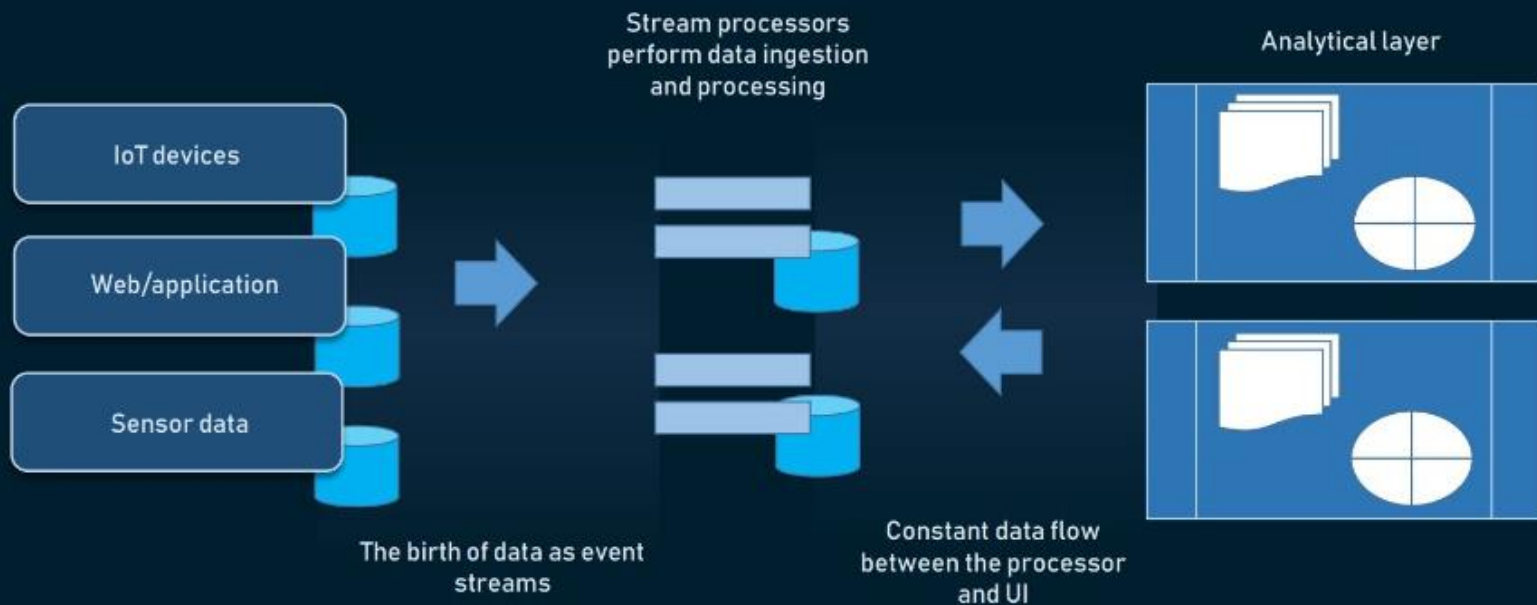
BATCH DATA PROCESSING ARCHITECTURE



Stream Processing



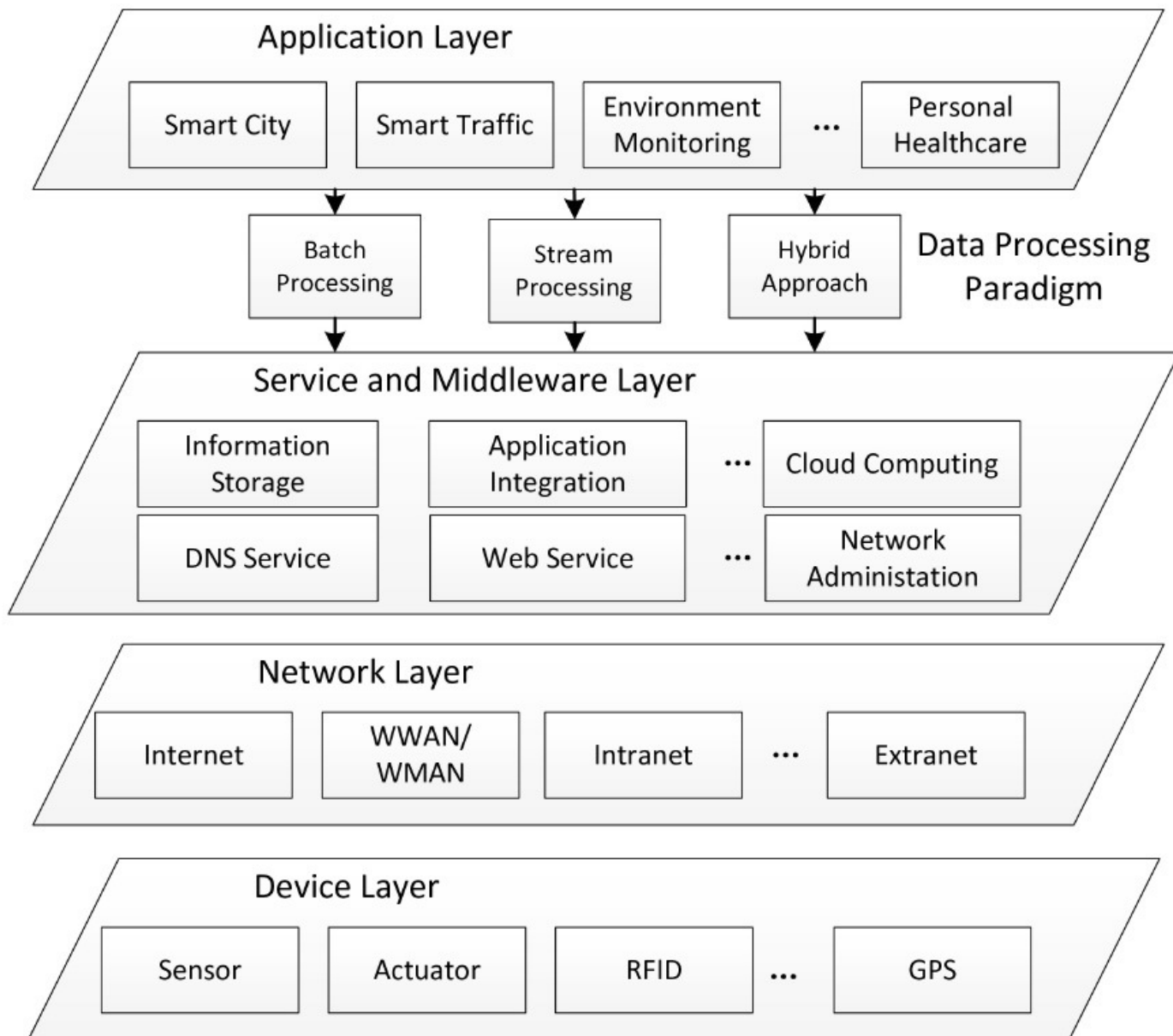
STREAM DATA PROCESSING ARCHITECTURE



- Stream processing is proposed as an alternative of the batch paradigm
- Caters to the need of processing continuous data volume in real-time
- Both data aggregation and analysis have a strict deadline specified
- Completing the job beyond the deadline is considered as a failure
- The streaming paradigm has to continuously aggregate the target-data elements right after its generation to form possible endless streams over the network

- When it comes to the data processing phase, these streams flow through a computation topology where **continuous queries** (ie, long-standing queries that usually operate over time and buffer windows) **are installed to be processed in a record-by-record manner**
- The streaming model essentially **deals with the dynamic data-streams that had recently comes in, and it incrementally updates the query results**

- Those data that have passed the processing system cannot be easily retrieved
- Stream processing has always been an integral part of Internet of Things (IoT) applications
- Stream processing mainly works as a connecting bridge between the application layer and the service and middleware layer
- Allows the upper logic to make appropriate use of the underlying general- purpose services and infrastructures



- For example, the streaming paradigm may decide that **only the synopsis of incoming data needs to be preserved in the storage system so that no external database is required**
- Has a significant impact on the organization of the network layer and the device layer to keep that real-time promise
- **The major part of latency between the data generation and result delivery lies in the data-collection phase rather than the processing phase**

THE FOUNDATIONS OF STREAM PROCESSING IN IoT

- There is a considerable ambiguity related to the terms “stream processing” and “stream analytics”
- Stream processing in the context of parallel processing
- Allows applications to better exploit computation parallelism using a combination of heterogeneous resources

Stream

- A stream is a sequence of data elements ordered by time.
- The structure of a stream could consist of discrete signals, event logs, or any combination of time-series data
- A data stream has an explicit timestamp associated with each element
- Stream is defined in the context of IoT, as a Data Element–Time pair (s, Δ) , where:

1. 's' is a sequence of data elements that are made available to the processing system over time
- Data element may consist of several attributes, but it is normally atomic
- Data elements can be either regularly generated by sensor networks or randomly produced by real-world events such as user clicks on a website

2. 'Δ' is a sequence of a timestamp that denotes the sequence of data elements
- Timestamp is necessary to reconstruct the logic sequence
 - Timestamps can be also used to evaluate the real-time property of a stream-processing system

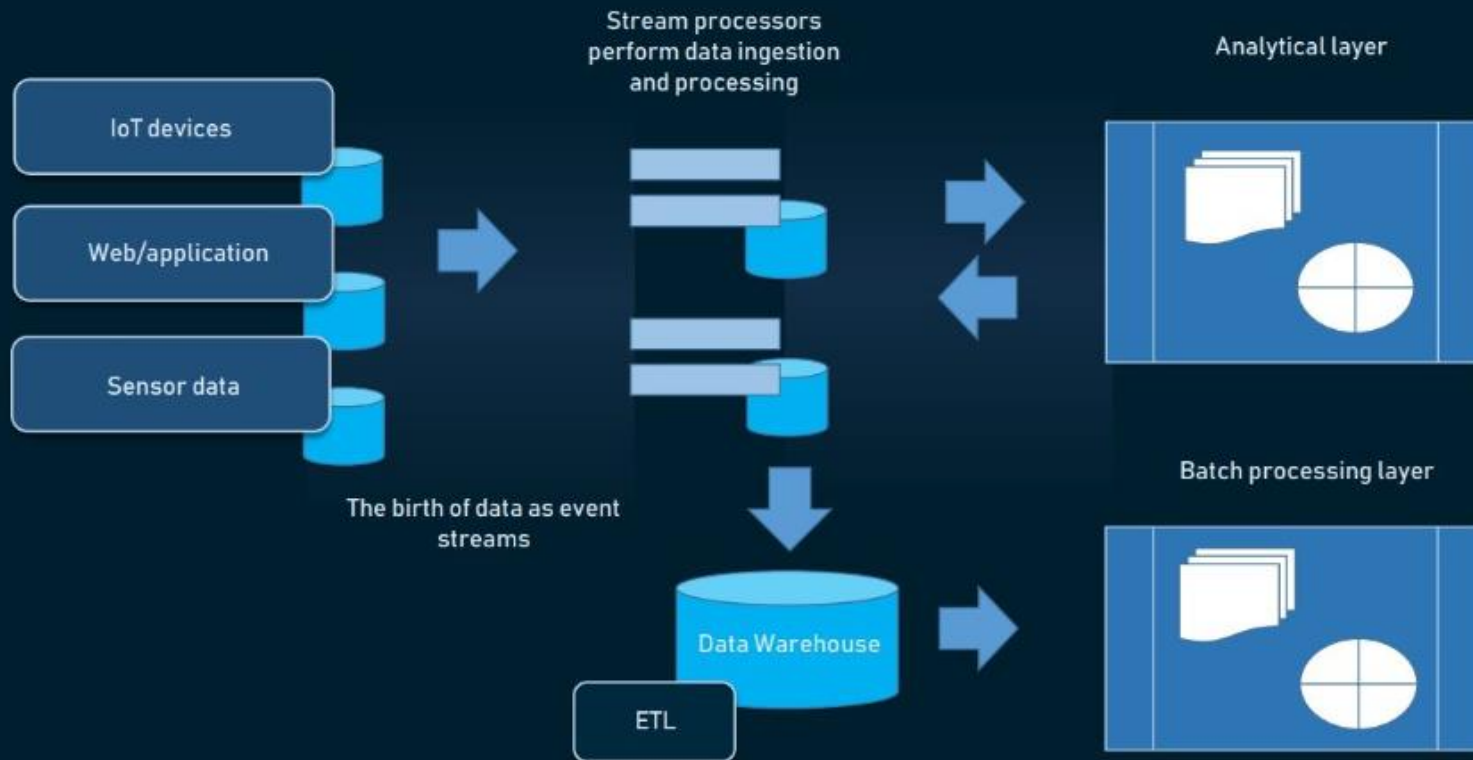
- Timestamps can be implemented in two forms:
 - ① As a **string of absolute time values**, which consumes more resources to be processed,
 - ② As a **sequence of positive real-time intervals** that only record the relative order of data elements in the same stream

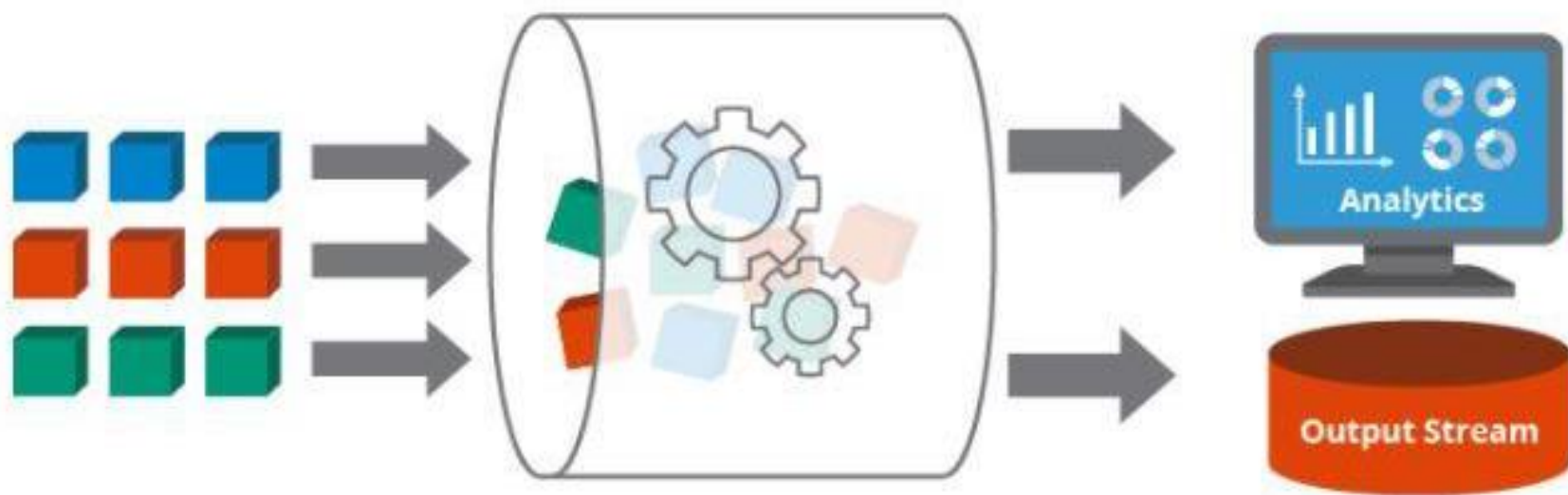
STREAM PROCESSING

- Stream processing is a one-pass data-processing paradigm that always keeps the data in motion to achieve low processing-latency
- Capable of performing real-time asynchronous computation while passing along the information

- Important feature of the streaming paradigm is that **it does not have access to all data**
- Adopts the one-at-a-time processing model, which applies **standing queries** or established rules **to data streams** in order to get immediate results upon their arrival
- There is another subclass of stream-processing systems that follows the **microbatch** model

STREAM PROCESSING COMBINED WITH BATCH PROCESSING





Input Data

**Stream Processing
Engine**

Outputs

- The microbatch model excels in controllability as a hybrid approach, combining a one-pass streaming pipeline with the data batches of very small size.
- Application of stream processing have two utterly different types of use cases
- The first one is *Data Stream Management*
- The system falling in this category is normally called Data Stream Management System (DSMS), analogous to the traditional DBMS

- Goal is to manipulate a huge amount of available data to constitute data synopsis, schema, or some other mathematical or statistical model that is easy to understand and interpret
- Data streams within the DSMS are joined, filtered, and transformed according to specific application logic
- A typical use-case of DSMS includes face recognition from a continuous video stream, and the calculation of user preference according to his or her click history.

User/Application

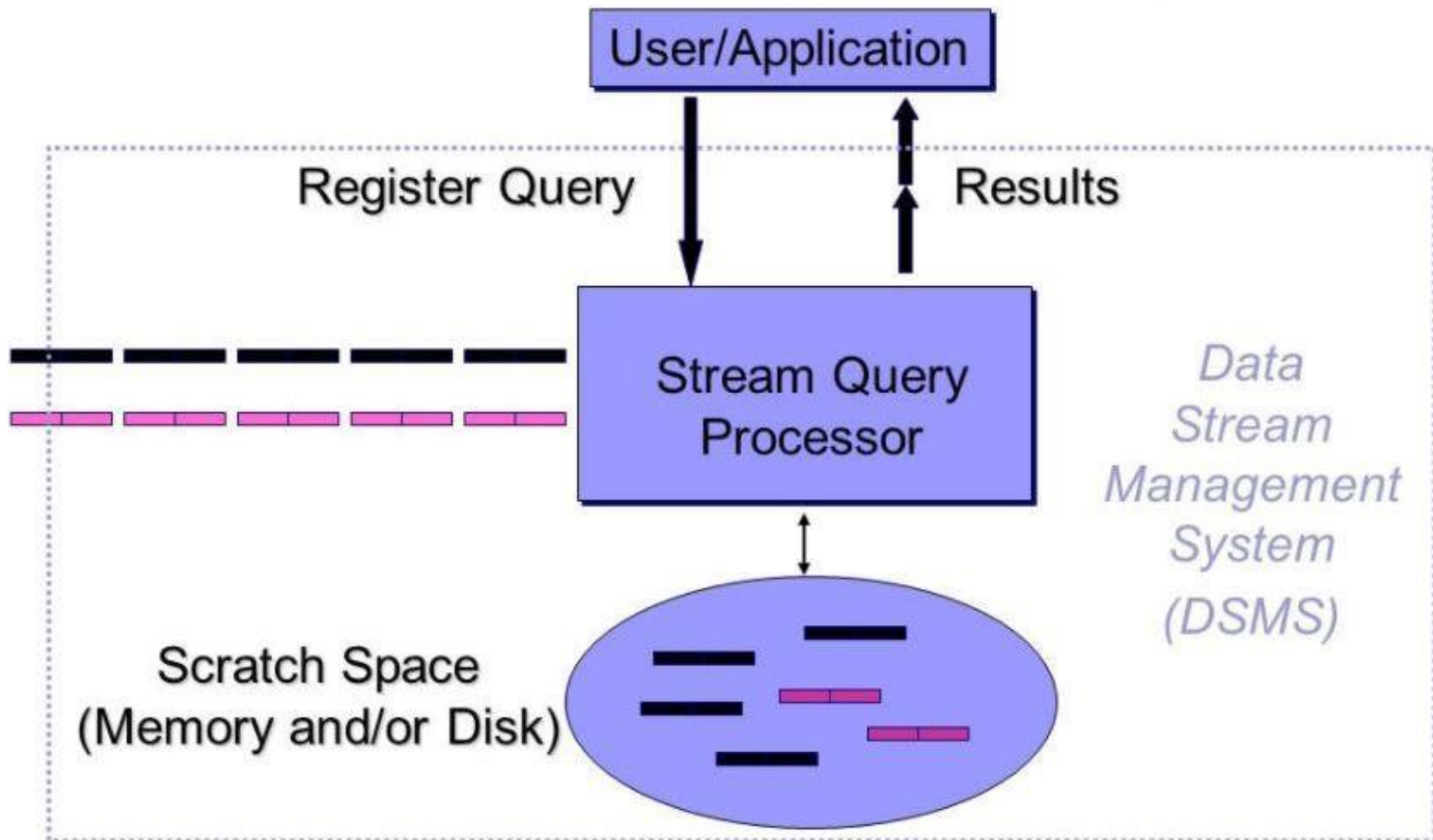
Register Query

Results

Stream Query Processor

*Data
Stream
Management
System
(DSMS)*

Scratch Space
(Memory and/or Disk)



Data Management Vs. Stream Management

- In DBMS input data is under the control of the programming staff
- Eg. SQL Insert Command
- Stream management is required when the input rate is controlled externally
- Eg. Google Search queries

Types of Queries in Stream

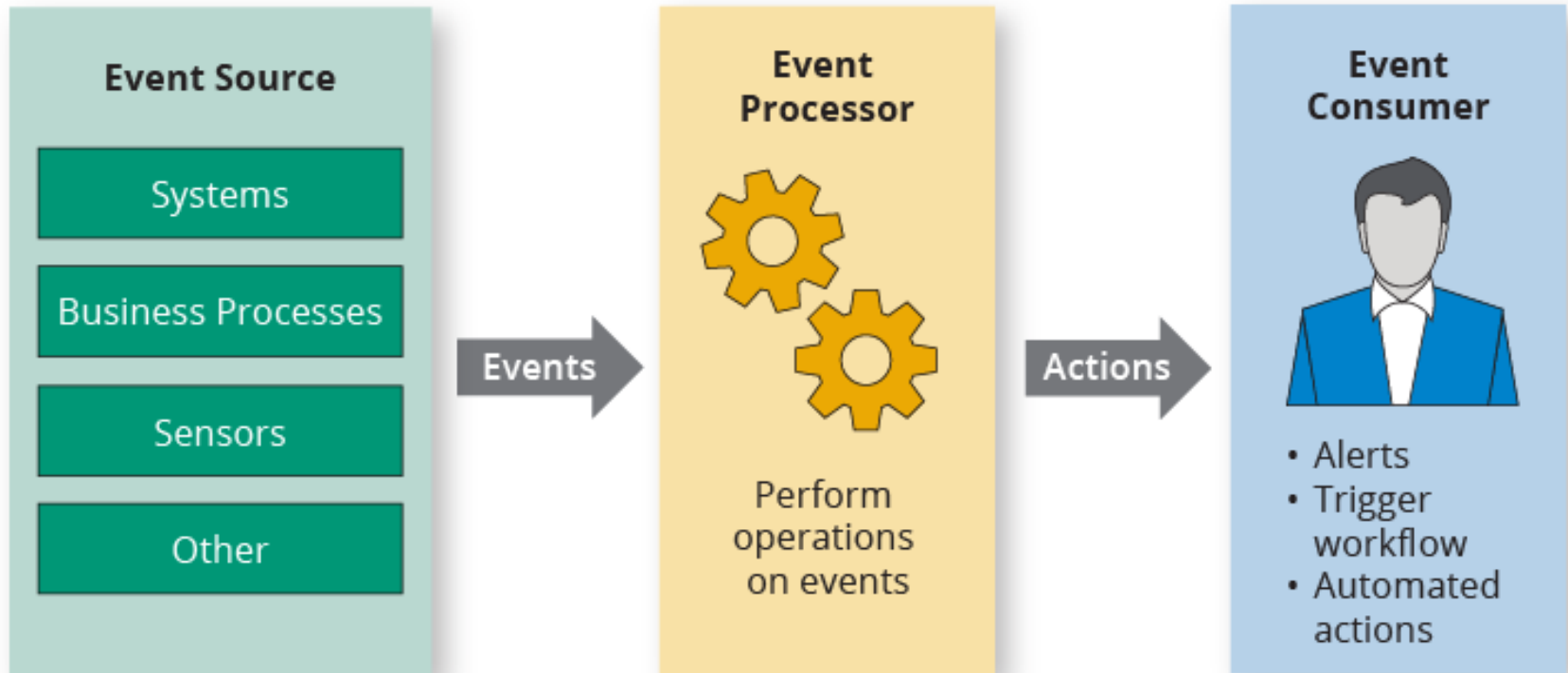
1. **Ad-hoc queries** – Normal queries asked **one time** about stream
 - Eg. What is the maximum value seen so far in stream 'S'
2. **Standing queries** – Queries asked about the stream at **all times**
 - Eg. Report each new maximum value ever seen in stream 'S'

Applications

- Mining query streams
 - Google wants to know what queries are more frequent today than yesterday
- Mining click streams
 - Yahoo! Wants to know which of its pages are getting unusual number of hits in the past hour
- IP packets monitoring at a switch
 - Gather optimal routing information
 - Detects DoS attacks

- The other use case is called **Complex Event Processing (CEP)**
- **Tracking and processing streams of raw events** in order to derive significant events and **identify meaningful insights from them**
- There are several techniques being used to achieve that goal
- The most notable one is **to implement and configure the processing logic as a set of inferring rules** in the knowledge base so that they could be used in the decision-making process of identifying complex patterns

- CEP systems normally require that the maintenance of state and the preservation of event relationship be provided at the system level rather than the application level
- This makes the microbatch model a preferable option compared to the one-at-a-time model
- Major concern of CEP is to infer the needed insight from the vast volume of raw events to stream as fast as possible
- Therefore, the computation complexity of CEP logic is usually lower than that of DSMS



Characteristics of Stream Data in IoT

1) Timeliness and Instantaneity

- Ensuring the timeliness of processing requires the ability to collect, transfer, process, and present the stream data in real-time
- The data generation in IoT environments mainly depends on the status of data sources
- It is necessary to build an adaptive platform that can elastically scale with respect to fluctuating processing demands, and still remain portable and configurable in response to the continuously shifting processing needs

2) *Randomness and Imperfection*

- Consequences of the dynamic nature of stream data
- The data generation process may induce randomness because the data sources are normally independently installed in different environments, which makes it nearly impossible to guarantee the sequence of data arrival across different streams
- The data transmission process can also result in disorder and other defections, as some tuples may be lost, damaged, or delayed due to the constantly changing network conditions

3) Endlessness and Continuousness

- As long as the data sources are alive and the stream-processing system is properly functioning, newly generated data will be continuously appended to the data channel until the whole application is explicitly turned off

4) Volatility and Unrepeatability

- Most of the stream data will be discarded once they have finished traversing through the stream processing system, which makes the existence of data quite volatile
- Even if the data sources are able to replay the data stream upon the retransmit request, the new stream is unlikely to be exactly the same as the previous one

Table 8.3 Characteristics of Stream Data and the Corresponding Processing Requirements

Characteristics	Corresponding Requirement
Timeliness and instantaneity	1. Data cannot be detained in any phase of the processing chain, so there should be a comprehensive data-collection subsystem working as a driving force that powers the data in motion once they are generated. 2. For compute-intensive applications, a data aggregation subsystem is needed to gather the collected data for centralized processing. 3. Each phase of the processing chain is preferable to be horizontal scalable in order to keep pace with the fluctuated workload.
Randomness and imperfection	1. For cleansing and coordination purposes, data should be first buffered in a message subsystem before being processed. 2. A declarative or imperative CLPS is responsible for implementing the application logic and handling possible data-stream imperfections.
Endlessness and continuousness	1. The storage subsystem can only be used as an assistance component that preserves the data synopsis or the query results. 2. Ensuring the availability is one of the core design principles due to the continuousness of workload.
Volatility and unrepeatability	1. The data value and insights discovered from the streams should be immediately submitted to other services or presented to users through a presentation subsystem. 2. The fault-tolerance ability is another system design principle, as it is costly or even impossible to replay the incoming stream during the recovery of system failures.

THE GENERAL ARCHITECTURE OF A STREAM-PROCESSING SYSTEM IN IoT

- A stream-processing architecture should include an integral data-processing chain that covers the whole lifespan of data (from its generation up to its consumption)
- This architecture breaks down the whole data-processing chain into several stages according to the functionality and target
- Identified six separate streaming components which are responsible for data generation, collection, buffering, logic processing, storage, and presentation, respectively

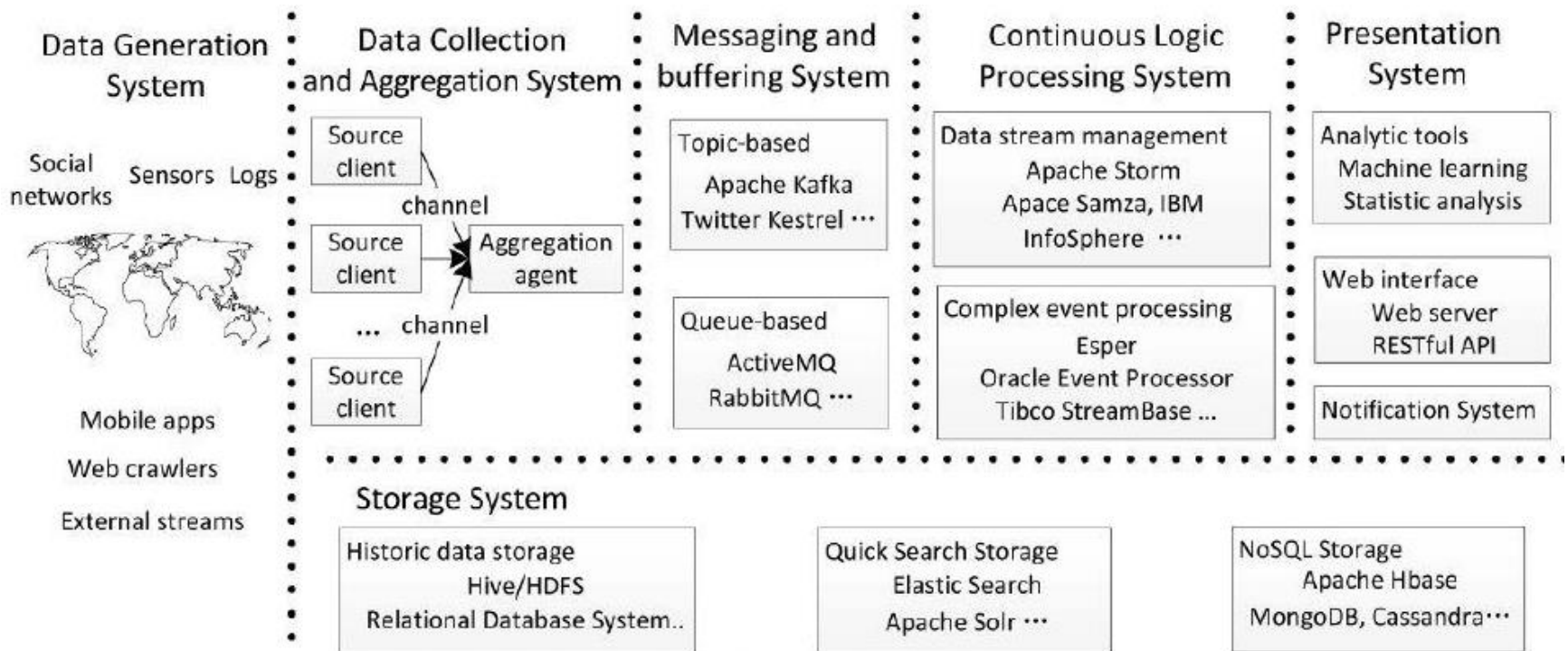


FIGURE 8.2 General Architecture of a Stream Processing System in IoT

1) Data generation System

- The data-generation system denotes the spectrum of data sources that continuously produce raw information for the data-processing chain
- We can still categorize the generated data into three types
- *Static data*, refers to the long-term information that has already been stored in remote locations

- ***Centralized stream data***, is a special type of stream that only comes from a single centralized data source
- ***Distributed stream data*** is the most common data type used in IoT applications, where data of this type dynamically come from various distributed places in heterogeneous formats, such as sensory information from sensor networks

2) Data collection and aggregation system

- To collect and aggregate different types of data, various forms of source clients are independently installed, while several aggregation channels are provided to gather these stream data into a centralized buffer, using hierarchical aggregation agents

3) Messaging and Buffering System

- Plays the role of a message broker in the whole data-processing chain
- There are two types of message buffers; some are **topic-based**, which support a higher-level programmability, whereas the others are **queue-based**, and thus mainly optimized for performance concerns

4) Storage System

- Supportive components for a stream-processing architecture
- Keeping all the data, that need to be stored are either **established knowledge**, which can guide the future processing, or meaningful **data synopsis**, which might arouse the future interest of users

5) Continuous Logic Processing System (CLPS)

- It is responsible for processing aggregated data according to the designated continuous logic
- Which could either come from the Data Stream Management or Complex Event Processing background

6) Presentation System

- Supportive components for a stream-processing architecture
- Serves as an interface for immediately hands over the data value to the higher level analytic tools, or directly delivers the results or notifications to the end users
- It is also responsible for receiving search-command or query updates from the external environment so that it can make the stream-processing system more adaptive and responsive

Continuous Logic Processing System (CLPS)

- The origin of the CLPS dates back to the beginning of this century
- The first generation of CLPS, pioneered by NiagaraCQ and STREAM is only suited for certain processing scenarios in which **only a small amount of data are generated**
- In addition to that, the **types of operations supported by these prototypes are also limited**

- They are usually used as functional extensions of the existing Data Base Management Systems (DBMS)
- NiagaraCQ defines a simple command-language to create and drop continuous queries over XML files
- STREAM directly supports SQL-like query language

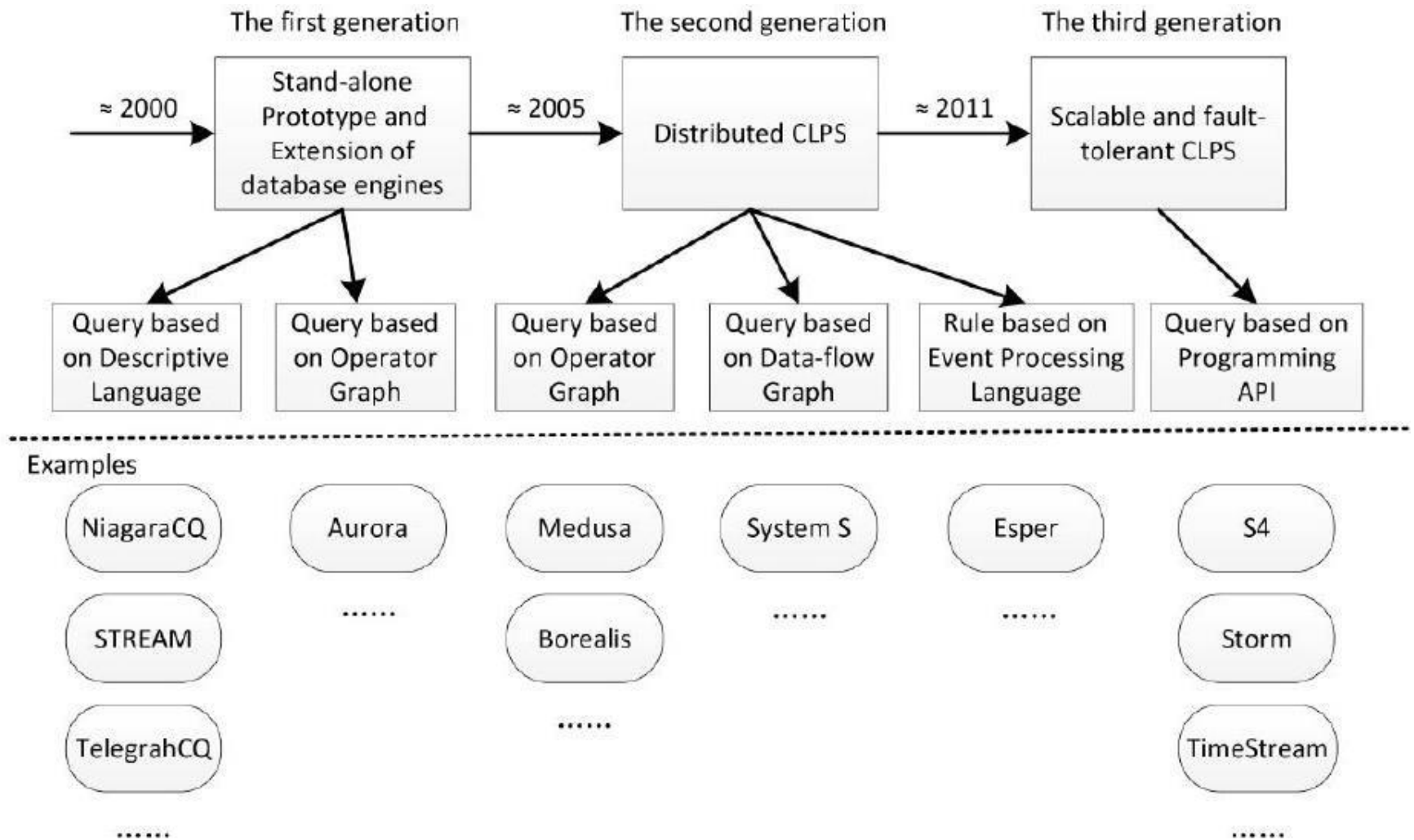


FIGURE 8.3 Evolutionary History of CLPS

- TelegraphCQ is developed on Postgre SQL to cope with the high value and diverse data streams, and enables the possibility of adaptive querying
- Aurora is the last breakthrough founded in the first generation of CLPS, here the continuous queries are implemented by explicit operator graphs rather than declarative query languages such as SQL
- The project Medusa is an extension to Aurora, which leads to a scalable and QoS-oriented architecture

- Borealis engine was developed on top of Medusa in order to integrate some advanced capabilities, including dynamic query modification, result revision, and flexible monitoring
- System S developed by IBM, proposed a query model based on data-flow graph to hide the implementation details as much as possible
- The core objective is to achieve highly scalable, resource-efficient processing with a balanced resource-allocation mechanism
- Esper is suitable for distributed event processing that has different types of events defined

- S4 developed by Yahoo! is generally perceived as the first CLPS that meets the criteria of fully scalable and fault-tolerant
- Storm supports almost arbitrary programming languages such as Clojure, Java, Ruby, and Python to implement the spouts and bolts, which are the logical operations in Storm
- TimeStream written in C# by Microsoft, it is able to handle an aggregation data-source with a data generation speed of 700,000 URLs per s

Challenges in Stream Processing Systems

- The current stream-processing systems have been greatly improved to cater to the emerging needs of IoT applications
- A stream-processing system now should satisfy the following criteria:
 - 1) Horizontal scalability to accommodate different sizes of processing needs
 - 2) Easy to program and manage
 - 3) Capable of dealing with possible hardware faults

- The following aspects summarize the challenges that still need to be further addressed
 - ✓ Scalability – scale up
 - ✓ Robustness – effectiveness
 - ✓ SLA-Compliance – level of service expected
 - ✓ Load Balancing – distribute the load

- **Scalability:**
- Scalability does not just refer to the ability to expand the system to catch up to the ever-increasing data streams, so that the promise of the Quality of Service (QoS) or Service Level Agreement (SLA) could be honored
- Elasticity, the ability to dynamically scale to the right size on demand, is the future and advanced form of scalability
- The user requirement may change over time, the system should dynamically provision new resources by taking into account the characteristics of the available hardware infrastructure, and free up some of them when they are no longer needed

- **Robustness:**
- **Fault-tolerance** is a commonplace topic when it comes to the design and implantation of stream processing systems
- Its **availability is one of the most crucial prerequisites** to guarantee the correctness and significance of real-time processing
- Designing a hybrid and configurable fault-tolerance mechanism that is capable of **recovering the system from unforeseeable failures** is an open research-question left to be answered

- **SLA – Compliance:**
- It depends on which platform the system is running on, and how stakeholders are involved
- But an inherent requirement is to achieve cost-efficiency, which minimizing the monetary cost for the users, as well as reducing the operational cost for the provider (possibly data centers)

- **Load Balancing:**
- The major target of which is to normally improve the performance of the system, especially by maximizing the throughput
- A wrong balancing decision may lead to unnecessary load-shedding, dropping arrived messages when the system is deemed to be overloaded, which impairs the accuracy of the processing result

Framework For Distributed Data Analysis For IoT

- Data are distributed in the network, and edge devices have limited capacity in terms of memory and computational power (Context)
- The main idea of this framework is to support efficient local processing and summarization of the data at the nodes, followed by global processing of the local summaries

- This framework reduces the amount of data routed over the network using the computational capabilities of the devices collecting the data (similar to Fog)
- Covers three aspects:
 - Local data modeling
 - Communication
 - Global data modeling

- The local data modeling requires an efficient algorithm with low computational cost.
- The algorithm calculates local summaries of the data and identifies anomalies in the local data.
- Only the summaries of the data are communicated to a central location which we refer to as *sink*
- The global data modeling component, which is located in a central location, is responsible for finding global summaries and anomalies

Preliminaries

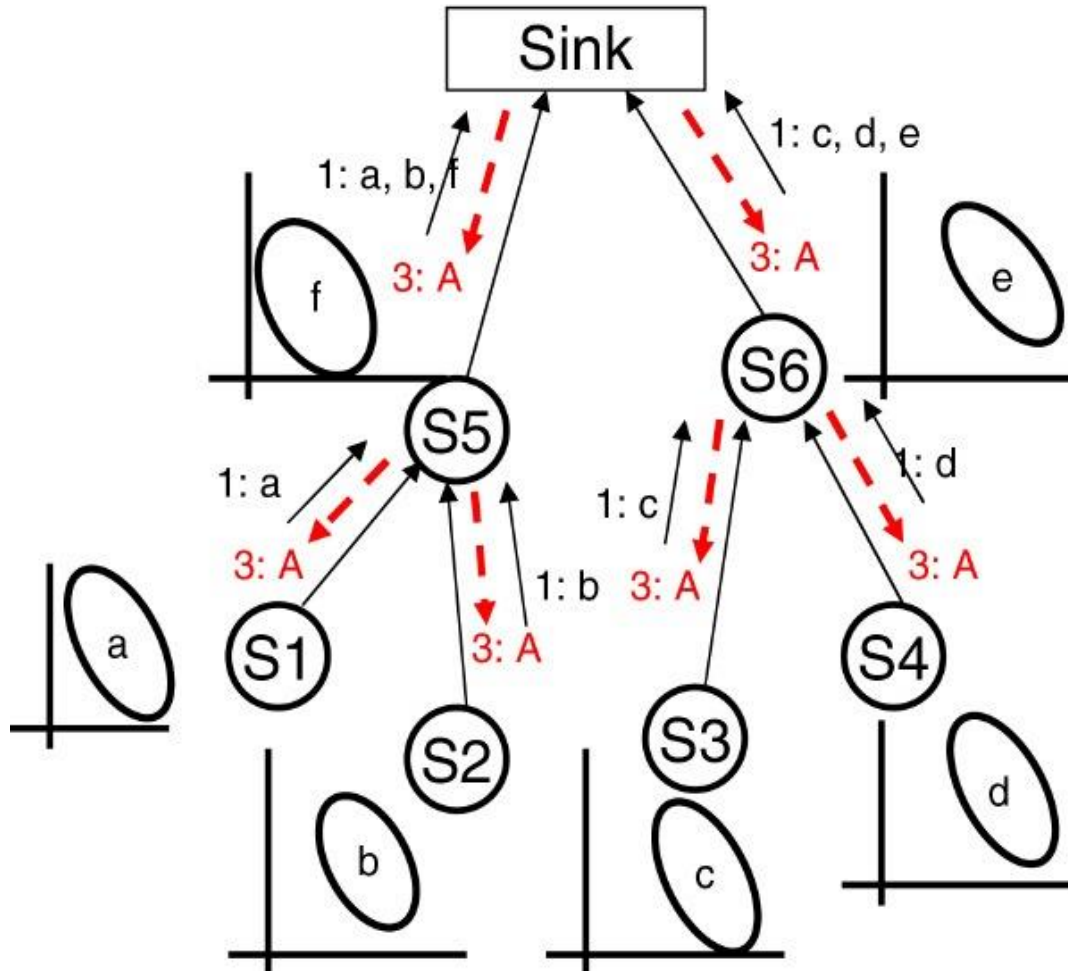
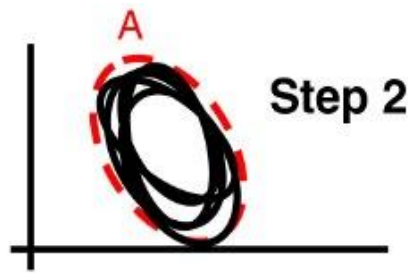
- Recent advancements in sensing technologies provide a cost-effective platform for monitoring applications to gather detailed observations from different environments including factories, mines, agriculture, and urban areas
- An important aspect of a monitoring system is the ability to detect significant events or unusual behavior in the environment

- These unusual patterns are also called outliers, surprises, novelties, or events in different applications
- Anomaly detection methods play an important role in modeling and detecting these anomalous events in IoT applications
- Numerous factors affect the use of anomaly detection such as mobility in sensors, the condition of the environment, the dynamics of the environment, and energy constraints

- The most common approach in anomaly detection is to build a model of the normal data and then identify deviations from this model as anomalies
- There are two components in the distributed framework: a local hyperellipsoidal model and distributed anomaly detection model

Distributed Framework

- In the distributed framework each node constructs a hyperellipsoidal decision boundary for its local data (local model) and sends the parameters of this decision boundary (step 1) to a sink node where all the local models are merged to find a global decision boundary for the network (step 2)
- The parameters of the global decision boundary are then sent back to the nodes (step 3)



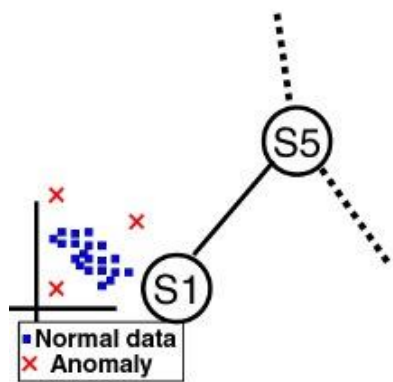
Anomaly Detection

- Anomaly detection is an important **unsupervised data processing task** which enables us **to detect abnormal behavior** **without having a priori knowledge** of possible abnormalities
- An anomaly can be defined as a **pattern in the data** that does not conform to a well-defined notion of normal behavior

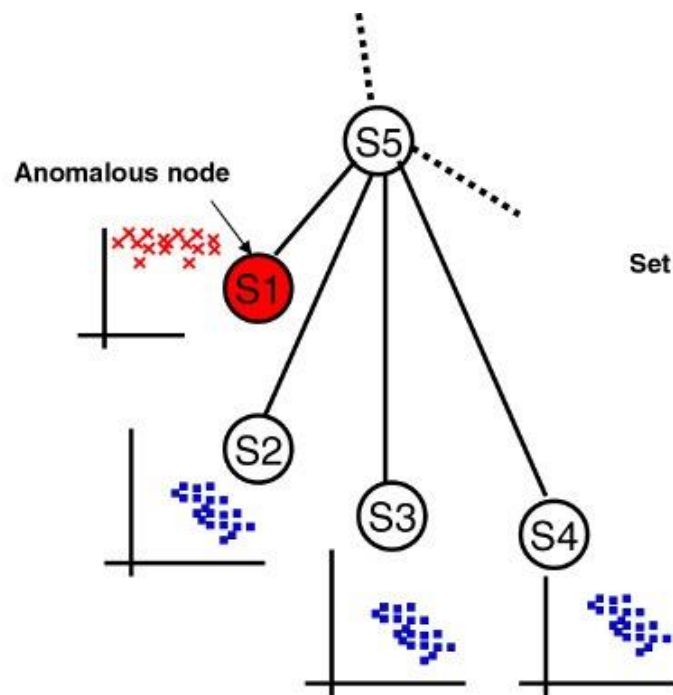
- Based on how patterns deviate from normal behavior
- We can categorize anomalies in the data into three categories:
 - **Outliers**—Short **anomalous patterns** that appear in a non-systematic way in the collected data, usually **arising due to noise or faults**, for example, due to communication errors.
 - **Events/Change**—These patterns appear with a systematic and **sudden change from previously known normal behavior**.
 - The duration of these patterns is usually longer than outliers.

- Environmental monitoring, extreme weather conditions are examples of events.
- The start of an event is usually called a change point.
 - **Drifts**—Slow, unidirectional, long-term changes in data.
- This usually happens due to the onset of a fault in a sensor
- Another categorization of anomalies is considering the topology of a network.

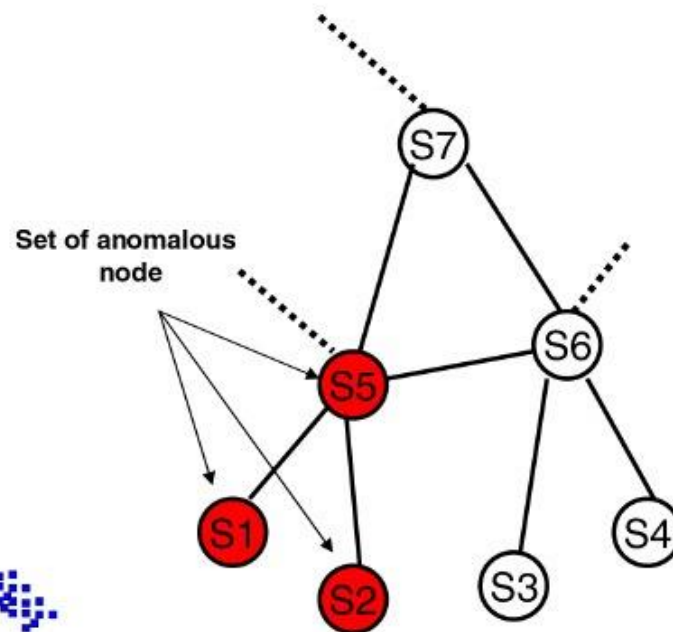
- Three categories of anomalies identified in this study are as follows:
 - **First order**—Anomalies can occur in an individual node, that is, some observations at a node are anomalous with respect to the rest of the data.
 - **Second order**—All of the observed data at a node can be anomalous with respect to the neighboring nodes.
- In this case, such a node is considered as an anomalous node in the network.
- **Third order**—A cluster of nodes are anomalous with respect to other nodes in the network.



First order anomalies



Second order anomalies



Third order anomalies

- The first type of anomaly detection techniques uses **rule-based** methods
- These techniques have been successfully implemented in many applications such as **intrusion detection**
- If an observed data sample does not match a predefined set of rules, it is considered as an anomaly
- The next type of anomaly detection approaches use **dynamic system modeling to model the normal behavior of the data**, and anomalous behavior is then identified by the extent of deviation from the normal model of the data